

Final Project : Snake Game

邓语乐 2351273

同济大学 2023 级 计算机科学与技术学院 软件工程专业 嵌入式系统方向 汇编语言课程作业

授课教师：王冬青

授课学期：2025-2026 年度 秋季学期

授课学期：2025-2026 年度 秋季学期

目录

- [项目介绍](#)
- [项目代码分析](#)
- [项目开发流程](#)
- [技术难点与解决方案](#)
- [编译与运行](#)
- [项目总结](#)

项目介绍

项目概述与背景

贪吃蛇游戏是一个经典的电子游戏，玩家控制一条蛇在屏幕上移动，通过吃食物来增长身体和获得分数。游戏的目标是尽可能获得高分，同时避免蛇撞到边界或自己的身体。

用汇编语言实现贪吃蛇游戏是一个很好的练习项目。这个项目涉及到了中断处理、数据结构管理、游戏逻辑实现等多个方面。通过使用键盘中断（INT 09h）来处理方向键输入，可以实现实时的游戏控制。虽然汇编语言的代码比较繁琐，但通过合理的数据结构和模块化的程序设计，可以实现一个功能完整的游戏。

功能模块列表

模块名称	功能描述
MAIN	主程序入口，控制游戏流程
showWelcome	显示欢迎界面和操作说明
waitForStart	等待用户按空格键开始游戏
initGame	初始化游戏数据（蛇的位置、食物等）

模块名称	功能描述
<code>installKeyboardInterrupt</code>	安装键盘中断处理程序
<code>keyboardHandler</code>	键盘中断处理程序（核心）
<code>moveSnake</code>	根据方向移动蛇
<code>checkCollision</code>	检查碰撞（边界和自身）
<code>checkFood</code>	检查是否吃到食物
<code>generateFood</code>	生成随机位置的食物
<code>drawGame</code>	绘制游戏画面
<code>showGameOver</code>	显示游戏结束界面
<code>uninstallKeyboardInterrupt</code>	卸载键盘中断处理程序
<code>clearScreen</code>	清屏
<code>printNumber</code>	打印数字（十进制）
<code>printString</code>	打印字符串
<code>newLine</code>	换行
<code>gameDelay</code>	游戏延迟（控制速度）

项目代码分析

数据段 (.DATA)

```
.DATA
snakeX dw 200 dup(0)    ; 蛇身X坐标数组（最多200节）
snakeY dw 200 dup(0)    ; 蛇身Y坐标数组
snakeLength dw 3        ; 蛇的长度
direction db 1          ; 方向：0=上，1=右，2=下，3=左
foodX dw 0              ; 食物X坐标
foodY dw 0              ; 食物Y坐标
gameOver db 0           ; 游戏结束标志
score dw 0              ; 分数
gameSpeed db 8          ; 游戏速度
```

- `snakeX` 和 `snakeY`：使用两个数组来存储蛇身每一节的坐标。每个数组最多可以存储 200 节，这样可以支持很长的蛇。
- `snakeLength`：记录当前蛇的长度，初始值为 3。
- `direction`：记录蛇的移动方向，使用 0-3 的数值表示上下左右四个方向。

- `foodX` 和 `foodY`：存储食物的位置坐标。
- `gameOver`：游戏结束标志，1 表示游戏结束。
- `score`：玩家得分，每吃一个食物加 1 分。
- `gameSpeed`：控制游戏速度，数值越大游戏越慢。

键盘中断处理

这是这个项目最核心的部分。通过拦截键盘中断（INT 09h），可以实现实时的方向控制。

```
keyboardHandler PROC FAR
    pushf                ; 保存标志寄存器
    push ax              ; 保存寄存器
    push ds

    ; 设置数据段（中断处理程序需要访问代码段中的数据）
    push cs
    pop ds

    ; 从键盘端口60h读取扫描码
    in al, 60h           ; AL = 扫描码（按下时<80h, 释放时>=80h）

    ; 检查按键状态：最高位为1表示释放，为0表示按下
    test al, 80h         ; 测试最高位（80h = 10000000b）
    jnz checkEscRelease  ; 如果是释放，跳转检查ESC释放

    ; 防抖处理：避免同一按键被重复处理
    cmp al, cs:lastKeyProcessed
    jne checkKeys        ; 新按键，继续处理
    jmp normalKey        ; 重复按键，忽略

checkKeys:
    cmp al, 48h          ; 上箭头
    jne checkRight
    jmp arrowUp
    ; ... 其他方向键检查
```

键盘中断处理程序的主要功能：

1. 读取键盘扫描码（从端口 60h）
2. 判断是否是方向键按下（只处理按下，不处理释放）
3. 防抖处理，避免同一个按键被重复处理
4. 防止反向移动（比如当前向下，不能直接向上）
5. 更新方向标志

代码段 (.CODE) 分析

主程序流程

```

MAIN PROC
    mov ax, @data            ; 初始化数据段寄存器
    mov ds, ax

    call showWelcome         ; 显示欢迎界面和操作说明
    call waitForStart        ; 等待用户按空格键开始游戏
    call initGame            ; 初始化游戏数据
    call installKeyboardInterrupt ; 安装键盘中断处理程序

    ; ===== 游戏主循环 =====
gameMainLoop:
    cmp gameOver, 1          ; 检查游戏是否结束
    je endGame

    cmp cs:escPressed, 1     ; 检查是否按下ESC键
    je endGame

    ; 从中断处理程序获取最新方向
    mov al, cs:directionTSR
    mov direction, al

    call moveSnake           ; 根据方向移动蛇
    call checkCollision      ; 检查是否撞墙或撞到自己
    call checkFood           ; 检查是否吃到食物
    call drawGame            ; 绘制游戏画面
    call gameDelay           ; 延迟控制游戏速度

    jmp gameMainLoop         ; 继续下一帧

endGame:
    call uninstallKeyboardInterrupt ; 恢复原键盘中断向量
    call showGameOver        ; 显示游戏结束界面
    mov ah, 4ch              ; 退出程序
    int 21h
MAIN ENDP

```

主循环逻辑：

1. 检查游戏结束条件
2. 获取最新方向（从中断处理程序）
3. 移动蛇
4. 检测碰撞
5. 检测食物
6. 绘制画面
7. 延迟控制速度
8. 循环继续

蛇的移动算法

```

moveSnake PROC
    push ax
    push bx
    push cx
    push dx
    push si

    ; 从中断处理程序获取最新方向
    mov al, cs:directionTSR
    mov direction, al

    ; 获取当前蛇头位置
    mov ax, snakeX[0]      ; 蛇头X坐标
    mov bx, snakeY[0]      ; 蛇头Y坐标

    ; 根据方向计算新蛇头位置
    cmp direction, 0        ; 0=上
    je moveUp
    cmp direction, 1        ; 1=右
    je moveRight
    ; ... 其他方向

moveUp:
    dec bx                  ; Y坐标减1 (向上)
    jmp moveDone

moveRight:
    inc ax                  ; X坐标加1 (向右)
    jmp moveDone

; ... 其他方向

```

移动算法的实现思路使用了比较直观的算法：

1. 根据当前方向计算新的蛇头位置
2. 从蛇尾开始，每一节移动到前一节的位置
3. 更新蛇头位置
4. 检查是否超出边界

碰撞检测

```

checkCollision PROC
    ; 获取蛇头位置
    mov ax, snakeX[0]
    mov bx, snakeY[0]

    ; 检查蛇头是否与蛇身重叠 (从第2节开始, 第1节紧邻蛇头不算碰撞)
    mov cx, snakeLength      ; 蛇的总长度

```

```

    dec cx                ; 跳过蛇头
    mov si, 2             ; 从第2节开始检查 (索引从1开始, si=2表示第2节)

checkSelf:
    cmp si, cx            ; 是否检查完所有节
    jg collisionOK        ; 检查完毕, 无碰撞

    shl si, 1             ; 乘以2 (word类型)
    ; 比较蛇头与当前节的坐标
    cmp ax, snakeX[si]    ; X坐标是否相同
    jne nextCheck
    cmp bx, snakeY[si]    ; Y坐标是否相同
    jne nextCheck

    ; 坐标相同, 撞到自己了
    mov gameOver, 1
    jmp collisionEnd

```

碰撞检测包括两部分：

1. 边界检测：检查蛇头是否超出屏幕范围
2. 自身碰撞检测：检查蛇头是否与蛇身重叠

食物是否吃到

原理与碰撞检测类似：比较蛇头坐标与食物坐标是否完全相等。若相等则认为“吃到食物”，触发得分增长、蛇身在尾部增长一节、并重新生成食物（同时调整游戏速度）。

判定方法：仅比较蛇头 (snakeX[0], snakeY[0]) 与食物 (foodX, foodY) 的 X、Y 坐标是否相等，属于 $O(1)$ 操作（与自碰撞检测不同，自碰撞需要遍历蛇身是 $O(n)$ ）。

触发效果：吃到食物要做三件事——score++、snakeLength++（并把新节位置初始化为尾部前一节的位置）以及调用 generateFood 生成新食物（generateFood 会确保新食物不与蛇身重叠，否则重试）。

食物生成

食物生成使用系统时间作为随机数源，如果生成的位置与蛇身重叠，就重新生成。

画面绘制

```

drawGame PROC
    ; ---- 在第0行显示分数 ----
    mov ah, 02h            ; 设置光标位置 (第0行第0列)
    mov bh, 0
    mov dx, 0
    int 10h                ; 此处省略：具体寄存器组合详见原代码

```

```

    lea dx, scoreMsg          ; 显示 "Score: "
    mov ah, 09h
    int 21h                  ; 此处省略: DOS 字符串输出的细节
    mov ax, score
    call printNumber          ; 显示分数数字 (printNumber 过程保留)

; ---- 绘制蛇身 (遍历所有节) ----
    mov cx, snakeLength
    mov si, 0                ; 从第0节 (头) 开始

drawSnake:
    push cx
    shl si, 1                ; 索引乘以 2 (word 类型偏移)
    mov ax, snakeX[si]        ; X 坐标 -> AX
    mov bx, snakeY[si]        ; Y 坐标 -> BX
    ; 设置光标到 (AX,BX)
    ; 此处省略: INT 10h 功能02h 的寄存器赋值/调用细节 (将 DL/DH 置为列/行并调用)
    push dx                  ; 保存光标寄存器内容
    ; 判断是蛇头还是蛇身并显示相应字符
    cmp si, 0
    je drawHead
    mov dl, '*'              ; 蛇身字符
    jmp drawChar

drawHead:
    mov dl, '@'              ; 蛇头字符

drawChar:
    mov ah, 02h              ; 显示字符 (INT 21h 功能02h)
    int 21h
    pop dx                   ; 恢复保存的光标寄存器
    shr si, 1                ; 恢复索引值 (除以2)
    inc si                   ; 下一个节
    pop cx
    loop drawSnake

; ---- 绘制食物 ----
    mov ax, foodX
    mov bx, foodY
    ; 设置光标到 (foodX, foodY), 此处省略, INT 10h/INT 21h 的具体寄存器赋值与调用
    mov dl, '#'              ; 食物字符
    mov ah, 02h
    int 21h                  ; 显示食物字符 (细节省略)
    ; 恢复寄存器并返回 (省略)
drawGame ENDP

```

绘制函数使用 BIOS 中断 (INT 10h) 来设置光标位置, 然后用 DOS 中断 (INT 21h) 来输出字符。蛇头用 '@' 表示, 蛇身用 '*' 表示, 食物用 '#' 表示。

项目开发流程

阶段一：需求分析与设计

明确功能需求（蛇的移动控制、食物生成与检测、碰撞检测、分数与速度控制）和非功能需求（中断响应、模块化与可读性、性能）。设计数据结构（使用 snakeX/snakeY 两数组存坐标）、方向编码与必要的状态变量，并按功能划分模块：主程序、中断处理、游戏逻辑、显示与工具函数。

阶段二：核心功能实现

搭建程序框架并实现主循环与基础工具（清屏、输出等）。完成键盘中断处理：保存/恢复中断向量、读取扫描码、识别方向键并加防抖与反向保护。实现游戏逻辑：方向计算与从尾到头的身体移动、边界与自身碰撞检测、食物随机生成与重叠检测及相应处理。

阶段三：功能完善与优化，以及测试与调试

完善界面（欢迎与结束画面、分数显示），实现并调优分数、增长与速度调整机制；重构并注释代码，优化结构与性能，修复已知边界或逻辑问题。对游戏进行功能测试（方向控制、碰撞、食物、分数）、边界与极端情形测试（边界碰撞、自身碰撞、长蛇情形、快速按键）并依据测试结果修复问题、优化速度与中断防抖逻辑。

阶段四：文档编写

为代码添加完整注释与关键算法说明，编写项目说明、代码分析、使用指南与开发总结，整理成果材料以便提交与答辩。

技术难点与解决方案

难点 1：键盘中断处理

刚开始写键盘中断处理程序的时候，遇到了很多问题。主要是：

- 中断处理程序必须保护所有寄存器
- 需要正确确认中断（向 8259 发送 EOI）
- 要防止按键被重复处理

解决方法是添加了防抖机制，记录上次处理的按键，如果是同一个按键就忽略。另外，方向键释放时会清除记录，允许下次按下。

难点 2：蛇身移动算法

蛇身移动的实现比较麻烦，因为需要从尾部开始，每一节移动到前一节的位置。刚开始我尝试从头部开始移动，结果发现会覆盖数据。后来改成从尾部开始，问题就解决了。

数组索引的计算也要注意，因为坐标是 word 类型，所以索引要乘以 2（使用 shl 指令）。

难点 3：食物生成避免重叠

因为食物不能生成在蛇身上，所以我使用系统时间作为随机数，用来检查生成的位置是否与蛇身重叠，如果重叠就重新生成。虽然理论上可能无限循环，但实际使用中很少出现这种情况。

难点 4：游戏速度控制

游戏速度的控制也是一个问题。刚开始速度太快，后来通过多层循环延迟来解决。吃食物后速度会加快，我改成了每吃 5 个食物才加快一次，这样速度变化更平缓。

编译与运行

使用 MASM 编译：

```
masm SNAKE.asm;  
link SNAKE.obj;
```

运行：

```
SNAKE.EXE
```

游戏操作

1. 运行程序后显示欢迎界面
2. 按空格键开始游戏
3. 使用方向键（↑↓←→）控制蛇的移动
4. 吃食物（#）来增长和得分
5. 按 ESC 键退出游戏

游戏特点

- 实时响应：使用键盘中断，按键立即生效
- 防抖处理：避免按键重复处理
- 碰撞检测：边界和自身碰撞检测
- 动态难度：随着分数增加，游戏速度逐渐加快（但变化很平缓）
- 分数系统：实时显示当前分数

项目总结

对于汇编语言的学习

这个项目让我对汇编的理解有了很大提升。实践中最直观的收获是：

中断处理能力有所提高：通过编写并安装 INT 09h 的键盘中断处理程序，我学会了如何在底层拦截按键并实现实时响应；用两个数组存蛇身坐标，让我熟悉了在汇编中如何操作数组。尤其是 word 类型的数组索引需要乘以 2 这一点，开发中反复验证才明白其必要性。

汇编语言和高级语言的不同在写蛇的碰撞、移动等算法中更有体会：这些算法在高级语言里写起来很直观，但在汇编中每一步都要写清楚，处理边界和索引的细节工作量明显更大，这个过程锻炼了我的底层实现能力；同时在开发中遇到了不少问题（比如跳转超出、类型不匹配等），通过定位和修复这些问题，我也获得了宝贵的调试经验。

收获

通过这个项目，我不仅掌握了汇编语言的基本编程技巧，更重要的是理解了：

- 中断机制的工作原理
- 底层系统编程的特点
- 如何用汇编语言实现复杂的逻辑

虽然汇编语言写起来比较繁琐，但它提供了对硬件的直接控制，这是高级语言无法做到的。这个项目让我体会到了汇编语言在系统编程中的独特价值。

可能的改进方向

1. **添加高分记录**：可以将最高分保存到文件中，下次运行时显示。
2. **多级难度**：可以设置不同的难度级别，对应不同的初始速度。
3. **更好的画面**：可以使用图形模式，绘制更美观的画面。
4. **音效**：可以添加吃食物和游戏结束的音效（虽然之前尝试过，但 DOSBox 中可能不支持）。

项目总结

这个项目相比简单的游戏程序（比如猜数字；因为我最开始想做的是猜数字的项目：系统给出一个 0-99 的随机数，我们进行猜测，系统告诉我们数字猜大了还是小了，我们再次猜测，共十次机会，猜对即停），技术含量更高：

- 使用了中断处理技术，这是汇编语言的重要特性
- 实现了完整的游戏逻辑，包括移动、碰撞、食物生成等
- 代码量约 900 多行，结构清晰

虽然项目还有一些可以改进的地方，比如可以添加高分记录、多级难度等，但基本功能都已经实现了。通过这个项目让我对汇编语言有了更深入的理解，还提升了汇编语言的编程能力，也理解了底层系统编程的一些概念。虽然过程有些困难，但最终完成了一个功能完整的游戏，还是很有成就感的。